

CMPT 280

Tutorial: Review of the Comparable interface.

Mark G. Eramian

University of Saskatchewan

Review: Comparable

- `Comparable<T>` is an interface in the `java.lang` package.
- If an instance of an object `A` implements `Comparable<T>`, it means that `A` is an object that knows how to compare itself to an instance of type `T` using a method called `compareTo` which takes a parameter of type `T`. Let's look at the API...
- Thus, when we want to be able to test if an instance is equal to another instance, the instances need to be of classes that implement the `Comparable` interface. But, you could write your own comparison methods without using the `Comparable` interface. Question: why use the interface?
- Remember: we can't just use comparison operators – that will only compare *references* (and determine if two references refer to the same instance), not the contents of the object!

Review: Comparable – Examples

Example

```
1 /**  
2  * A class X whose instances know how to compare other instances of X to itself.  
3  */  
4 public class X implements Comparable<X>
```

Because instances of *X* *and* subclasses of *X* all have the same instance variables common to *X*, it is possible to compare instances of *X* not only to other instances of *X* but also to *subclasses* of *X*.

Review: Comparable – Examples

It is also possible to define a class Y that can compare itself to instances (or instances of subclasses) of *another* class Z to itself.

Example

```
1  /**
2   * A class Y whose instances know how to compare other instances of Z,
3   * and any instances that are subclasses of Z, to itself.
4   */
5  public class Y implements Comparable<Z>
```

However, this is much less common than classes that know how to compare instances of *themselves*.

Review: Comparable – Examples

In any case, if a class implements `Comparable<T>` it must have a method of the form

```
public int compareTo(T other)
```

where `other` is the other instance to compare to itself.

Example

```
1  /**
2   * A class X whose instances know how to compare other instances of X,
3   * and any instances that are subclasses of X, to itself.
4   */
5  public class X implements Comparable<X> {
6      /**
7       * Compare this object to an instance of X.
8       * @param other The other object to compare to.
9       * @return negative integer if this object < other, positive integer if
10         this object > other, 0 if they are equal.
11      */
12     public int compareTo(X other) { ... }
13
14 }
```

Review: Comparable

- If we wanted to extend a simple `LinkedList` (like the one we did in class) to be an ordered list ADT (with additional operations), we would need a way to ensure that the type parameter `T` implements the `Comparable` interface.
- Do you remember how to do this?

Ordered List of Comparables – First Attempt

```
1 public class OrderedSimpleList<I extends Comparable<I>>
2                                     extends LinkedList<I>
3 {
4     /* Implementation... */
5 }
```

This will work for many cases, but it's not as general purpose as it could be. What's wrong with this class declaration? Let's see a motivating example...

An Ordered List of Books

Any object that we want to put in our ordered list must meet the requirements for type parameter `I` of `OrderedSimpleList`. Here is the information about books that we want to store in an ordered list. It's straightforward to make a `Book` class that contains this information with accessors and mutators.

Book
isbn : String # author: String # title : String
+ Book(t : sString, a : String, i : String) + getTitle() : String + setTitle(t : String) + getAuthor() : String + setAuthor(a : String) + getIsbn() : String + setIsbn(i : String)

An Ordered List of Books

Supposing we've implemented `Book`, now we'll see how to implement a `ComparableBook` object which implements the `Comparable` interface so that books can be stored in the ordered list.

Pay close attention to the interface that our `ComparableBook` class will implement, the class that it extends, and the `compareTo` method within the `Book` class.

Ordered List Elements

```
1 // A Book that can be compared to another Book by comparing their isbn numbers.
2 public class ComparableBook extends Book implements Comparable<ComparableBook>
3 {
4     /**constructor */
5     public ComparableBook(String t, String a, String i)
6     {
7         super(t, a, i);
8     }
9
10    /**Is the current Book less than 'other'?
11     * @timing Time = O(1) */
12    public int compareTo(ComparableBook other)
13    {
14        return isbn.compareTo(other.isbn);
15    }
```

Ordered List Elements

- Since `ComparableBook` implements `Comparable<ComparableBook>` it has to have a method:

```
public int compareTo(ComparableBook other)
```
- Note how this method uses the ISBN number to impose an order on the books. Since the ISBN number is stored as a `String`, we can use the `compareTo` method of the `String` class to determine the result (since `String` implements `Comparable<String>`).
- Now `ComparableBook` meets the requirements of the type parameter `I` of `OrderedSimpleList`.
- So far, so good. But what happens when we create a subclass of `ComparableBook`?

BetterComparableBook

```
1 // A subclass of the ComparableBook, with more fields.
2 public class BetterComparableBook extends ComparableBook
3 {
4     int pages;
5
6     /**constructor */
7     public BetterComparableBook(String t, String a, String i, int p)
8     {
9         super(t, a, i);
10        pages = p;
11    }
12
13    public int getPages(){
14        return pages;
15    }
16 }
17 }
```

BetterComparableBook

- BetterComparableBook inherits all the methods (including compareTo) from ComparableBook.
- BetterComparableBook also extends the Comparable<ComparableBook> interface.
- But, that doesn't match the generic type required by our OrderedSimpleList: <I extends Comparable<I> >. To put a BetterComparableBook into an OrderedSimpleList we would need BetterComparableBook to extend Comparable<BetterComparableBook>

Two solutions:

- 1. We could change the class definition to `BetterComparableBook` implements `Comparable<BetterComparableBook>`, and add an `int compareTo(BetterComparableBook other)` method.
 - But, this introduces a lot of coding if we have many subclasses. Further, the `compareTo` method in `BetterComparableBook`, might just call the `compareTo` in `ComparableBook`!
- 2. We could change the type requirements on `OrderedSimpleList`.
 - Much more interesting!

Ordered List of Comparables – Second Attempt

```
1 public class OrderedSimpleList<I extends Comparable<? super I>>
2                                     extends LinkedList<I>
3 {
4     /* Implementation ... */
5 }
```

- The type parameter for the list is I, which means that the list stores instances of class I. But notice the restriction on I – I must extend Comparable<? super I>.
- We now have one version of the OrderedSimpleList that can be used store objects of any class that implements the Comparable interface, for their own class, or any of their superclasses!

Manipulating a list of books:

```
1 // This is illegal! (Why?)
2 OrderedSimpleList<Book> BookList = new OrderedSimpleList<Book>();
3
4 // This is OK.
5 OrderedSimpleList<ComparableBook> CBookList =
6     new OrderedSimpleList<ComparableBook>();
7
8 // Store a book in the ordered list.
9 CBookList.insertFirst(new ComparableBook("The_␣Odyssey", "Homer", "0140268863"));
10
11 // Print the first book in the list.
12 // (What is the type of CBookList.firstItem()? Why does this work?)
13 System.out.println( CBookList.firstItem() );
```